

RUI3 Security Node — Operator Manual

What this technology does

SEC is the separate RAK4631 security-test node used only for authorized LoRaWAN authentication/replay/spoofing fixtures in this project lab.

Project history calls the same physical role both SEC-01 and SEC-02. Do not treat those names as two separate nodes unless the topology is deliberately changed.

SEC must never receive EMU-01's legitimate AppKey or session keys.

Current accepted baseline

Hardware	RAK4631 on RAK19007
Firmware	RUI_4.2.4_RAK4631
Normal mode	LoRaWAN
Band	BAND=8 / AS923 family fixture baseline
Join mode	OTAA
Class	A
Parked joined state	not joined
Parked DevEUI	0000000000000000

Older RUI3 3.4.2 is historical; it lacked controls required by the accepted raw-RF fixture.

Safe parked state

```
NWM=1
BAND=8
NJM=1
CLASS=A
NJS=0
DEVEUI=0000000000000000
no EMU-01 AppKey
no EMU-01 session key material
```

Keep the LoRa antenna attached.

Step 1 — Identify the physical node

```
[ ] physical board is SEC
[ ] base is RAK19007
[ ] normal sensor/IO slots empty unless a specific fixture requires them
[ ] LoRa antenna attached
[ ] this is not EMU-01
```

Stop if identity is uncertain.

Step 2 — Discover the current COM port

```
Get-CimInstance Win32_SerialPort | Select-Object DeviceID,Description,PNPDeviceID
```

Historical accepted state used COM16, but reset/DFU may change it.

Step 3 — Pause recorder ownership if needed

```
$R = '.\lorawan-network-server-gateway\test\automation\research-recorder\research_recorder.py'; python $R  
serial-pause --device sec
```

After the approved action:

```
python $R serial-resume --device sec
```

Step 4 — Verify RUI3 identity read-only

At the AT console:

```
AT+VER=?  
AT+BUILDTIME=?  
AT+REPOINFO=?  
AT+HWMODEL=?  
AT+HWID=?
```

PASS means: the expected RAK4631/RUI3 family is present and the accepted current firmware is identified.

Save only sanitized output.

Step 5 — Verify parked state before transmission

Use current RUI3 read-only status queries to prove:

```
LoRaWAN mode  
BAND=8  
OTAA  
Class A  
not joined  
all-zero parked DevEUI
```

If SEC unexpectedly contains a production identity, stop before transmitting.

Step 6 — Prefer the reviewed fixture helper

Use:

```
test/automation/research-recorder/sec02_replay.py
```

The helper selects already-captured authorized evidence, enforces replay-age checks, correlates with gateway evidence, supports dry-run review, performs the approved SEC fixture action and restores the parked state.

Do not manually decrypt/re-encrypt or reconstruct EMU-01 traffic for replay.

Step 7 — Dry-run before RF transmission

```
python .\lorawan-network-server-gateway\test\automation\research-recorder\sec02_replay.py send --fixture '<RUN-DIR>\replay-R01\fixture.json' --dry-run
```

Review the fixture and restoration plan before a real authorized transmission.

Use the real transmit action only through a currently READY test procedure.

Step 8 — Require gateway reception

SEC reporting transmit success is not enough.

A counted RF security attempt is valid only when:

```
SEC transmits  
-> Gateway-01 / RAK5146 receives a distinct RF frame  
-> downstream security behavior can be classified
```

If RAK5146 did not receive it, the attack attempt is invalid and must be repeated according to the experiment manual.

Step 9 — Keep LoRaWAN security enabled

Do not disable or weaken:

```
frame-counter validation  
MIC validation  
OTAA key validation  
normal ChirpStack security behavior
```

The experiment measures the commissioned security controls.

Step 10 — Prove downstream outcome

For a gateway-received rejected frame, require recorder evidence that no new legitimate application event, telemetry row or Fabric outbox item was produced from that rejected frame.

Do not infer rejection only from Grafana.

Step 11 — Restore parked state after every fixture

Verify:

```
LoRaWAN mode  
BAND=8  
OTAA  
Class A  
not joined
```

Do not leave SEC in P2P/raw transmit mode between trials.

Step 12 — RUI3/DFU recovery

If SEC has the wrong firmware:

1. positively identify SEC;
2. use the approved RAK4631-R DFU/conversion procedure;
3. hash the selected package;
4. require Device programmed;
5. rediscover the application COM port;
6. verify RUI3 identity;
7. restore parked state;
8. perform only an uncounted reception rehearsal before counted testing.

Do not copy old package names blindly from historical screenshots.

Current accepted capability

RUI3 4.2.4 previously demonstrated the required raw-RF controls and one uncounted transmission was received by Gateway-01.

Each counted security attempt still requires its own gateway-reception proof.

Troubleshooting order

Symptom	First check
no SEC COM	USB / application-vs-DFU mode
AT command missing	wrong RUI3 version/family
serial busy	recorder/other terminal owns port
SEC TX, gateway no RX	antenna/RF parameters/current conditions
gateway RX, unexpected acceptance	preserve evidence; inspect frame/session/security state
SEC left in raw mode	restore parked state immediately
EMU-01 secret material present	stop; remove it and treat as trust-boundary violation

Security boundary

- authorized lab fixture only;
- never install EMU-01 legitimate AppKey/session keys;
- use dedicated temporary fixture identities where required;
- do not store secret keys in result files;

- dry-run first;
- prove RF reception;
- restore parked state after each fixture.

Completion checklist

- correct SEC hardware identified;
- current COM discovered;
- RUI3 4.2.4 verified;
- parked state verified;
- no EMU-01 secret material present;
- reviewed helper/fixture used;
- dry-run reviewed;
- gateway reception proven for counted attempts;
- recorder captures downstream outcome;
- parked state restored;
- evidence preserved unmodified.

Detailed references

- [Sensor/RUI3 preparation](#)
- [Replay/spoofing experiment](#)
- [Research recorder](#)

Supporting runbook: 01-configure-rak4631-emulators.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

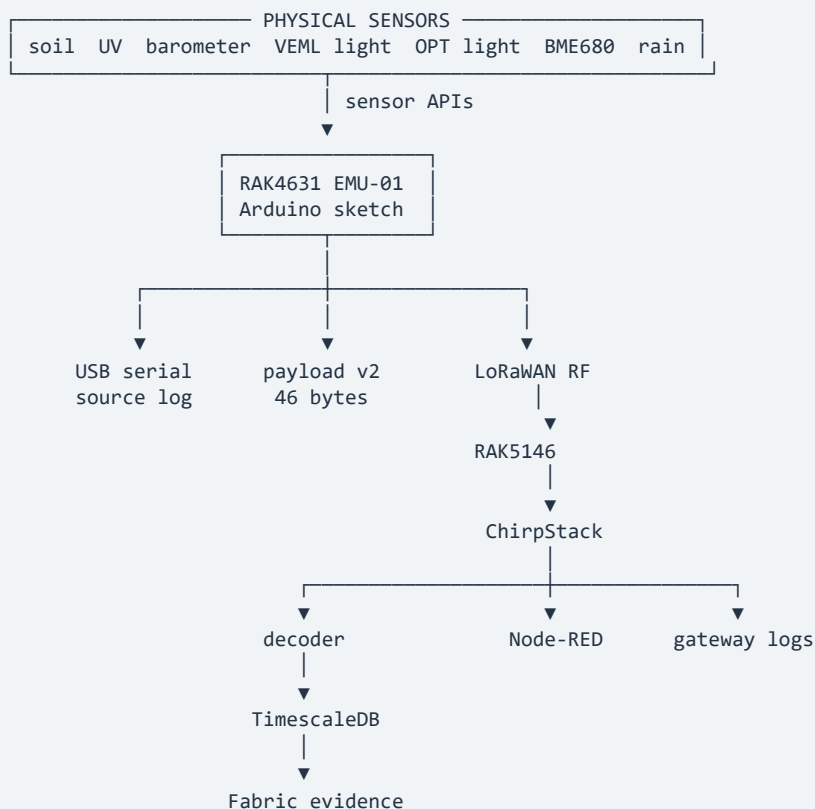
Sensor 1 - Program the RAK4631, Configure LoRaWAN, and Prove the Complete Sensor Path

The filename is retained so existing links do not break. EMU-01 is a real physical Agriculture Kit sensor node. SEC-02 is the separate sensor-verification/security node.

Use this manual **after** all hardware and individual-sensor checks in [assembly/04-verify-all-sensors.md](#) pass.

Architecture

Final EMU-01 path



Firmware lifecycle of the two cores

```
CORE A / EMU-01
Arduino RAK4631
├── individual sensor tests
├── integrated sensor bring-up
└── FINAL full-sensor LoRaWAN firmware

CORE B / SEC-02
Arduino RAK4631 on RAK19007
├── Profile A: A=RAK1903-B, B=RAK12010-B,
│           C=RAK12019-B, D=RAK12011-B,
│           IO=RAK12023+RAK12035-B
├── POWER COMPLETELY OFF
├── Profile B: A=RAK1906-B, B/C/D=EMPTY,
│           IO=RAK12005+RAK12030-B
├── save B-copy evidence
├── remove temporary Sensor/IO modules
└── only AFTER sensor evidence is complete
    │
    ▼
    stripped security baseline
    RAK19007 + Core B + LoRa antenna
    Sensor A-D = EMPTY; IO = EMPTY
    │
    ▼
    security-node firmware path
    RUI3 conversion if using AT/P2P fixtures
```

Do not convert SEC-02 to RUI3 before its B-copy sensor verification is complete unless you intentionally plan to convert it back to the Arduino firmware family for those tests.

Part A - Final EMU-01 firmware

Step 1 - Confirm the prerequisites

Do not continue unless:

```
[ ] EMU-01 = RAK19001 + Core A
[ ] all seven A-set sensor types installed
[ ] fixed RAK19001 map is A=RAK1903, B=RAK12010, C=RAK12019, D=RAK12011, E=RAK1906, F=EMPTY/RESERVE,
WisIO1=RAK12023, WisIO2=RAK12005
[ ] Pin Mapper result for that exact map is saved with no unresolved conflict
[ ] LoRa antenna attached
[ ] Arduino IDE installed
[ ] RAKwireless Arduino BSP installed
[ ] basic upload to Core A already proven
[ ] every A-copy sensor worked individually
[ ] ten-cycle integrated sensor bring-up already worked
[ ] every B-copy sensor has been or will be verified before SEC-02 security conversion
```

If a sensor cannot initialize in the integrated bring-up sketch, fix that before adding LoRaWAN logic.

Step 2 - Record and freeze the development environment

Create/update:

```
chapter4-results/_device-baseline/EMU-01-firmware.txt
```

Record:

```
Arduino IDE version
RAKwireless Arduino BSP version
selected board = WisBlock Core RAK4631 Board
sensor library versions
LoRaWAN/SX126x library version used by the selected RAK example
final source revision/hash
build date
```

Do not update libraries during a counted experiment group.

Step 3 - Open the final firmware project

Use one dedicated Arduino sketch/project for EMU-01.

Recommended logical layout:

```
EMU01_Agriculture_Node/
├──
├── EMU01_Agriculture_Node.ino
├── sensor_readers.h          # optional helper file
├── payload_v2.h             # optional helper file
└── secrets.h                 # LOCAL ONLY - never commit
```

secrets.h is only an example local organization. If another secure local provisioning mechanism is used, document it. The important rule is that the legitimate AppKey must not enter Git or result/evidence folders.

Step 4 - Select the board and USB port

In Arduino IDE:

1. select WisBlock Core RAK4631 Board;
2. connect only EMU-01 by USB-C data cable;
3. open Tools -> Port;
4. select the port that appears for Core A;
5. confirm the physical board label says EMU-01 before uploading.

Stop if: you are not certain which physical core the selected port belongs to.

Step 5 - Start from the known-good integrated sensor sketch

Do not begin by writing LoRaWAN code into an untested blank sketch.

Use the integrated bring-up sketch accepted in the previous manual as the base:

```
setup()  
├─ serial initialization  
├─ soil init  
├─ UV init  
├─ barometer init  
├─ VEML7700 init  
├─ OPT3001 init  
├─ BME680 init  
└─ rain init
```

Compile once before adding the LoRaWAN portion.

Expected result: the exact full-sensor sketch that already produced ten good sample cycles still builds.

Step 6 - Add the LoRaWAN code using a RAK4631 example as the reference

Use the LoRaWAN example/library supplied or referenced by the pinned RAK4631 BSP instead of writing the radio stack from scratch.

Before compiling LoRaWAN_OTAA_ABP, install the LoRaWAN dependency that provides LoRaWan-RAK4630.h:

```
Arduino IDE  
-> Sketch  
    -> Include Library  
        -> Manage Libraries...
```

Search for:

```
SX126x-Arduino
```

Install the SX126x-Arduino library, then close and reopen the sketch if Arduino IDE does not immediately refresh its include index. The RAK4631-specific header is supplied by this library. If compilation reports:

```
fatal error: LoRaWan-RAK4630.h: No such file or directory
```

the LoRaWAN library is missing or Arduino IDE is not seeing the installed copy; this is not a ChirpStack, OTAA-key, sensor, or radio-region failure.

The final application must provide:

```
OTAA  
Class A  
approved project AS923 configuration  
DevEUI  
JoinEUI/AppEUI  
AppKey  
60-second local sampling schedule plus 5-minute nominal production uplink schedule with fleet  
staggering/jitter  
unconfirmed/confirmed uplink choice frozen before testing
```

Use the LoRaWAN version actually implemented by the pinned build. Do not claim a different MAC version in ChirpStack or the dissertation.

Step 7 - Keep credentials out of the repository

Prepare locally:

```
LEGIT_DEV_EUI
LEGIT_JOIN_EUI
LEGIT_APP_KEY
```

Rules:

```
DevEUI/JoinEUI may be recorded as identifiers
AppKey must not appear in Git
AppKey must not appear in screenshots/results
EMU-01 AppKey must never be copied to SEC-02
EMU-01 session keys must never be copied to SEC-02
SEC-02 may temporarily have its OWN legitimate OTAA AppKey for its separate legitimate-node test
```

The temporary SEC-02 legitimate-node test must use a different DevEUI and different AppKey from EMU-01. After that test is recorded, its temporary legitimate credentials are retired before SEC-02 is converted into the security fixture.

If the AppKey is compiled into the sketch, keep its source definition in an ignored/local secrets file and archive only a **redacted** build record.

Part B - Payload v2

Step 8 - Freeze the binary layout

Final application payload:

Byte(s)	Field	Encoding
0	payload_version	uint8, fixed 2
1-4	test_sequence	uint32, big-endian
5-8	sensor_uptime_ms	uint32, big-endian
9-10	soil_moisture_percent_x100	uint16, big-endian
11-12	soil_temperature_c_x100	int16, big-endian
13-14	uv_index_x100	uint16, big-endian
15-18	barometer_pressure_pa	uint32, big-endian
19-20	barometer_temperature_c_x100	int16, big-endian
21-24	light_veml7700_lux_x100	uint32, big-endian
25-28	light_opt3001_lux_x100	uint32, big-endian
29-30	environment_temperature_c_x100	int16, big-endian
31-32	environment_humidity_percent_x100	uint16, big-endian
33-36	environment_pressure_pa	uint32, big-endian
37-40	environment_gas_resistance_ohm	uint32, big-endian
41	rain_wet	uint8, 0=dry, 1=wet
42-43	battery_mv	uint16, big-endian
44-45	sensor_validity_bitmap	uint16, big-endian

Total = **46 bytes**.

Do not silently change field order or size. Any incompatible layout change requires a new payload version and matching decoder.

UV field check

Before freezing payload v2, prove that the selected RAK12019 driver exposes the quantity you are calling `uv_index`. If it exposes only a different/raw UV quantity, use the correct documented field name and update both firmware and decoder before counted testing rather than mislabeling the value.

Battery field under USB-only testing

Do not fabricate a battery voltage when no battery measurement exists. If the final firmware cannot obtain a meaningful battery value during USB-only operation, use one documented sentinel consistently (for example 0) and state that rule in the payload baseline.

Step 9 - Implement the validity bitmap

```
bit 0 = soil valid
bit 1 = UV valid
bit 2 = barometer valid
bit 3 = VEML7700 valid
bit 4 = OPT3001 valid
bit 5 = RAK1906 environment valid
bit 6 = rain valid
bits 7-15 = 0
```

Healthy full cycle:

```
0b0000000001111111 = 0x007F
```

If a read fails:

```
read fails
├─ clear that sensor validity bit
├─ do NOT silently mark a stale previous sample as valid
└─ still keep this cycle's test_sequence
```

This prevents a sensor failure from being mistaken for an RF packet loss.

Step 10 - Implement one deterministic sample cycle

Use this exact logical order:

```
60-second local sample boundary
┆
┆   ┆
┆   ┆ refresh latest physical sample
┆   ┆
┆   ┆   ┆
┆   ┆   ┆ 5-minute nominal uplink deadline (DevEUI-staggered, ±15 s jitter)
┆   ┆   ┆
┆   ┆   ┆   ┆
┆   ┆   ┆   ┆ increment test_sequence
┆   ┆   ┆   ┆ ┆
┆   ┆   ┆   ┆ ┆   ┆ capture monotonic uptime
┆   ┆   ┆   ┆ ┆   ┆ clear validity bitmap
┆   ┆   ┆   ┆ ┆   ┆
┆   ┆   ┆   ┆ ┆   ┆ read SOIL -> set bit 0 on success
```

```
- read UV    -> set bit 1 on success
- read BARO -> set bit 2 on success
- read VEML -> set bit 3 on success
- read OPT  -> set bit 4 on success
- read ENV  -> set bit 5 on success
- read RAIN -> set bit 6 on success

- pack 46-byte payload
- print one serial source line
- request one LoRaWAN uplink
```

Do not increment `test_sequence` once per sensor. Increment it once per scheduled telemetry cycle.

Step 11 - Use the production fleet scheduler

The final EMU-01 firmware separates local sensing from RF transmission:

```
local physical sampling = every 60 seconds
normal uplink           = nominally every 300 seconds
initial uplink phase    = DevEUI-derived offset within the 5-minute window
per-cycle jitter        = ±15 seconds
normal uplink type      = unconfirmed
ADR                     = enabled
rain event poll         = every 1 second
rain event backoff      = random 0-5 seconds
rain event rate limit   = at most one event uplink per 60 seconds
```

The DevEUI-derived initial phase prevents a fleet that boots together from immediately transmitting together. Per-cycle jitter prevents nodes that drift together from remaining synchronized indefinitely. The implementation uses wrap-safe monotonic `millis()` deadlines instead of `delay(300000)`.

A rain-state transition may schedule one event uplink after randomized backoff. If a normal telemetry deadline is reached at essentially the same time, the normal uplink wins and the pending event is cleared so the node does not immediately transmit duplicate information.

Step 12 - Print one source line per cycle

Use a stable machine-readable format. Example:

```
SENSOR_TX,seq=125,uptime_ms=1875000,soil_pct=41.23,soil_temp_c=27.10,uv_index=0.18,baro_pa=100812,baro_temp_c=27.22,light_veml_lux=382.10,light_opt_lux=401.25,env_temp_c=27.44,env_humidity_pct=68.31,env_pressure_pa=100790,env_gas_ohm=92314,rain_wet=0,battery_mv=0,valid=0x007F,join=1,send_started=1,send_status=0
```

Use the actual measured values. The example numbers are formatting examples only.

The serial source log is authoritative for:

```
test_sequence
physical values sampled for that sequence
validity bitmap
scheduled transmission attempt
reset/rejoin evidence
```

Part C - Build and flash EMU-01

Step 13 - Compile before upload

In Arduino IDE:

1. selected board = RAK4631;
2. selected port = EMU-01;
3. verify required libraries are installed;
4. compile/verify the sketch;
5. resolve all build errors;
6. save the exact accepted source revision/hash.

Do not update random libraries just to make one compiler error disappear without recording the change.

Step 14 - Upload the final firmware

1. Confirm the LoRa antenna is connected.
2. Confirm EMU-01 is physically connected by USB.
3. Confirm you selected EMU-01's port.
4. Click Upload.
5. Wait for successful completion.
6. If the port cannot enter bootloader mode, close Serial Monitor, double-click reset, re-select the bootloader/new port if needed, and retry.
7. Reboot/reset EMU-01 after the successful upload.

Pass: the final sketch starts and prints its startup/sensor status to serial.

Step 15 - Verify local sensor behavior before joining LoRaWAN

Watch serial output and require:

```
all expected sensor initializations reported
RAK1906 stabilization requirement satisfied
soil calibration loaded/applied
both light values present separately
rain state readable
validity = 0x007F during healthy cycles
sequence increases once per actual LoRaWAN transmission attempt; local 60-second samples do not consume
sequence numbers
```

Capture at least ten final-firmware cycles over USB serial before troubleshooting the network.

If the validity bitmap is not 0x007F, fix the sensor first.

Part D - ChirpStack registration and OTAA

Current lab handoff - 2026-08-19

SEC-02 Profile A functional verification	= PASS (operator-confirmed)
SEC-02 Profile B functional verification	= PASS (operator-confirmed)
all seven B-copy sensor types	= FUNCTIONALLY VERIFIED
SEC-02 temporary legitimate DevEUI	= AC1F09FFFE296AEB
SEC-02 legitimate OTAA / ChirpStack connection	= PASS (operator-confirmed)
SEC-02 repeated 15-second application uplinks	= PASS (operator-confirmed)
SEC-02 RAK12011 radio telemetry path	= PASS to ChirpStack LoRaWAN-frame level
SEC-02 root AppKey	= configured locally; NOT recorded here
normal final legitimate ChirpStack node	= EMU-01
next SEC-02 action	= compare decoded RAK12011 values with Serial for several consecutive frames, then retire temporary legitimate mode

SEC-02 is now proven to complete OTAA and continue sending repeated real RAK12011 application uplinks through the RAK5146 gateway into ChirpStack. The accepted implementation sends the first sensor frame shortly after join and then continues on the 15-second application schedule from the normal Arduino loop() context. Keep SEC-02 on the Arduino firmware family only until the decoded temperature/pressure comparison is complete.

Never reuse EMU-01's AppKey or session keys. After the legitimate-node check fully passes, retire the temporary SEC-02 key/device activation before converting it to the security fixture.

Step 15A - Prove a minimal OTAA join before merging the full sensor payload

Do this once before the final 46-byte sensor firmware. It separates LoRaWAN/ChirpStack faults from sensor-integration faults.

On EMU-01:

```
LoRa antenna attached
↓
Arduino IDE
↓
File -> Examples -> LoRaWAN_OTAA_ABP
↓
configure AS923 + OTAA + Class A + credentials
↓
upload the stock Hello! payload
↓
prove JoinRequest -> JoinAccept -> first uplink in ChirpStack
↓
only then merge the seven-sensor payload v2 code
```

For the RAK LoRaWAN_OTAA_ABP example used by the standard RAK4631 Arduino BSP, configure the application values as follows:

```
LoRaMacRegion_t g_CurrentRegion = LORAMAC_REGION_AS923;
bool doOTAA = true;
lmh_confirm g_CurrentConfirm = LMH_UNCONFIRMED_MSG;
DeviceClass_t g_CurrentClass = CLASS_A;
#define LORAWAN_APP_INTERVAL 15000
```

Use unconfirmed uplinks for normal production telemetry. The final EMU-01 firmware uses a nominal 5-minute uplink interval with DevEUI-derived initial staggering and ± 15 -second jitter; the old 15-second cadence is retained

only in historical/bench procedures where explicitly stated. Do not use the default EU868 region.

Frozen lab region rule: the already-configured gateway and ChirpStack server use plain AS923 / region ID and MQTT prefix as923. EMU-01 must therefore use exactly LORAMAC_REGION_AS923. Do **not** use LORAMAC_REGION_AS923_3 for this lab unless the gateway and server are deliberately migrated at the same time.

The RAK example uses these OTAA arrays:

```
uint8_t nodeDeviceEUI[8] = { /* EMU-01 DevEUI, MSB order */ };
uint8_t nodeAppEUI[8]    = { /* EMU-01 JoinEUI/AppEUI, MSB order */ };
uint8_t nodeAppKey[16]   = { /* legitimate AppKey, LOCAL SECRET */ };
```

Do not paste real keys into this Markdown file. Keep the AppKey in a local ignored `secrets.h` or another approved local provisioning path.

ChirpStack 4.x click path for the minimal join

Before creating the device, verify the registered RAK5146 gateway shows a recent `Last seen`. A device cannot join through a gateway path that is not already healthy.

Create the device profile:

```
Tenant
-> Device Profiles
   -> Add device profile
```

Use:

```
Name: EMU-01 RAK4631 AS923
Region: AS923
Region configuration (if shown): as923 / plain AS923
Activation: OTAA
Device class: Class A
```

For the RAK Arduino example based on SX126x-Arduino, the library documents LoRaWAN MAC 1.0.2 and Regional Parameters 1.0.2 Rev B. Use those values when the ChirpStack form exposes them. If the pinned firmware/library is changed later, re-check and match the profile to the actual implementation rather than keeping these values by habit.

For the first join test, leave the payload codec disabled or use a raw/no-op codec. The stock RAK example sends `Hello!`; the Agriculture Kit 46-byte decoder is installed only after the join path is proven.

Create the application if it does not already exist:

```
Tenant
-> Applications
   -> Add application

Name: dissertation-sensors
```

Then create EMU-01:

```
Applications
-> dissertation-sensors
   -> Devices
      -> Add device
```

Use:

```
Name: dissertation-emu-01
DevEUI: <LEGIT_DEV_EUI>
Device profile: EMU-01 RAK4631 AS923
```

In ChirpStack v4, the normal local-join-server flow does not necessarily expose a separate JoinEUI field on the device form. The JoinEUI/AppEUI still exists in the OTAA JoinRequest and must match the value frozen in the EMU-01 firmware. After the device record is created, enter the legitimate root AppKey only on the protected OTAA keys page. Do not place it in screenshots or evidence files.

Minimal OTAA PASS gate

Open both of these ChirpStack views before resetting EMU-01:

```
EMU-01 -> LoRaWAN frames
EMU-01 -> Device data
```

Reset/power-cycle EMU-01. Require this order:

```
JoinRequest
↓
JoinAccept
↓
Join / activation event
↓
UnconfirmedDataUp or ConfirmedDataUp
↓
raw Hello! application payload appears
```

If the gateway sees JoinRequest but the EMU-01 device page does not, check the DevEUI. If ChirpStack sees the JoinRequest but rejects it, check AppKey, JoinEUI routing, device profile MAC/Regional Parameters, and AS923 configuration. If the minimal Hello payload works, freeze the LoRaWAN identity/profile and continue with the full sensor payload; do not change the radio settings while integrating sensors.

Step 15B - Temporarily test SEC-02 as its own legitimate sensor node

Use this stage when SEC-02 must first prove that its radio, OTAA implementation, gateway path, ChirpStack registration, and a real sensor payload all work under valid credentials.

Do this **before RUI3/security conversion**. Keep SEC-02 on the already-proven Arduino RAK4631 firmware family.

Identity separation

Use:

```
device name = dissertation-sec-02-legit
DevEUI      = unique SEC-02/Core-B DevEUI; MUST differ from EMU-01
JoinEUI     = the lab JoinEUI used by this local OTAA flow; all-zero is acceptable only if that is the
frozen lab convention
AppKey      = NEW 16-byte SEC-02-only secret
profile     = dedicated `SEC-02 RAK4631 AS923 LEGIT TEST` profile
class       = A
activation  = OTAA
```

Give SEC-02 its **own device profile** with the same LoRaWAN/region settings as the known-good Arduino RAK4631 profile. This keeps the temporary SEC-02 6-byte test codec isolated from EMU-01's 46-byte payload-v2 codec.

Use:

```
Tenant
-> Device Profiles
   -> Add device profile

Name: SEC-02 RAK4631 AS923 LEGIT TEST
Region: AS923
Region configuration: as923 / plain AS923
MAC version: 1.0.2
Regional Parameters: 1.0.2 Rev B
Device class: A
Codec: temporary SEC-02 6-byte barometer decoder
```

Then create a second device under the test application:

```
Applications
-> dissertation-sensors
   -> Devices
       -> Add device

Name: dissertation-sec-02-legit
DevEUI: AC1F09FFFE296AEB
Device profile: SEC-02 RAK4631 AS923 LEGIT TEST
```

Do not create SEC-02 by cloning EMU-01 credentials. Current temporary legitimate-node DevEUI: AC1F09FFFE296AEB.

After the device record is saved, open SEC-02's **Keys** (OTAA) tab and use ChirpStack's key-generation control to generate a new 128-bit AppKey for SEC-02. Save the generated key in ChirpStack, then copy that exact value only into SEC-02's local firmware secret/provisioning file. Do not paste the key into this repository, screenshots, evidence files, or chat. For LoRaWAN 1.0.x, this is the device's OTAA application root key even if an API/internal field is named `nwkKey`.

If a generated SEC-02 AppKey is accidentally disclosed in chat, screenshots, notes, Git, or evidence, treat that key as retired immediately: use ChirpStack to generate/replace it again before the first OTAA join and never record the exposed value in project documentation.

Fast radio-first check

First run the documented `LoRaWAN_OTAA_ABP` minimal uplink with SEC-02's credentials:

```
LORAMAC_REGION_AS923
OTAA = true
Class A
unconfirmed uplink
15-second interval
SEC-02 DevEUI
SEC-02 AppKey
```

Require:

```
JoinRequest seen by RAK5146
-> JoinAccept generated by ChirpStack
-> SEC-02 prints joined
-> at least one application uplink accepted
```

This isolates OTAA/radio configuration before adding a sensor read.

Real-sensor check

After the minimal join works, use the compile-ready **SEC-02 legitimate RAK12011 OTAA sensor sketch** in [assembly/04b-emu01-sec02-code-reference.md](#). Keep the verified RAK12011-B installed in Sensor D for this check.

Accepted scheduler rule: keep RAK12011 I2C reads, Serial printing, payload packing, and `lmh_send()` in the normal Arduino `loop()` execution path. Use a monotonic `millis()` deadline to trigger the first post-join uplink and each later 15-second attempt. Do not move the sensor read and `lmh_send()` into a `TimerEvent_t` callback. During bench troubleshooting, that callback-based design completed OTAA but stalled before normal application uplinks; the loop-based scheduler produced repeated `UnconfirmedDataUp` frames at the intended cadence.

The sketch sends a six-byte payload containing:

```
bytes 0-1 = RAK12011 temperature C x100, signed int16 big-endian
bytes 2-5 = RAK12011 pressure hPa x100, uint32 big-endian
```

Install the temporary decoder from that same code-reference section or inspect the raw bytes. Require multiple consecutive valid uplinks whose decoded temperature/pressure agree with SEC-02 Serial output within the fixed integer scaling.

Record only non-secret evidence:

```
SEC-02 legitimate OTAA = PASS/FAIL
SEC-02 DevEUI
join UTC window
JoinRequest observed = yes/no
JoinAccept observed = yes/no
number of consecutive real-sensor uplinks
sample serial temperature/pressure
sample decoded temperature/pressure
```

Do not save the AppKey.

Exit from temporary legitimate mode

After PASS:

1. stop SEC-02 transmissions;
2. record the legitimate-node result;
3. remove/disable the temporary dissertation-sec-02-legit device record when it is no longer needed, or at minimum rotate/retire its AppKey before security testing;
4. do not reuse its legitimate session state for attack fixtures;
5. power SEC-02 off before changing Sensor/IO hardware;
6. continue to Part H only when ready to strip the sensors and convert SEC-02 into the isolated security fixture.

Step 16 - Register EMU-01 in ChirpStack

Create one normal OTAA device using:

```
device name: dissertation-emu-01
DevEUI: <LEGIT_DEV_EUI>
JoinEUI/AppEUI: <LEGIT_JOIN_EUI>
```

```
activation: OTAA
class: A
region: plain AS923 (firmware `LORAMAC_REGION_AS923`; ChirpStack region `as923`)
LoRaWAN MAC version: match final firmware
target normal uplink interval: 5 minutes nominal, ±15-second jitter; local sample interval: 60 seconds
```

Enter the legitimate AppKey only in the approved ChirpStack credential field and the secure firmware/provisioning path.

Step 17 - Perform the first OTAA join

Observe simultaneously:

```
EMU-01 serial monitor
RAK5146/gateway evidence
ChirpStack device events
```

Expected sequence:

```
EMU-01 starts
  |
  ▼
JoinRequest transmitted
  |
  ▼
RAK5146 receives
  |
  ▼
ChirpStack validates OTAA
  |
  ▼
JoinAccept
  |
  ▼
EMU-01 joined
```

Stop if: sensor reads work but OTAA fails. Troubleshoot antenna, AS923 configuration, EUI/key values, device profile, and gateway reception rather than changing sensor libraries.

Part E - ChirpStack decoder

Step 18 - Install the payload-v2 codec

Use a dedicated codec that rejects incorrect versions/lengths:

```
function decodeUplink(input) {
  const b = input.bytes;
  if (!b || b.length !== 46) {
    return { errors: ["expected 46-byte Agriculture Kit payload v2"] };
  }
  if (b[0] !== 2) {
    return { errors: ["unsupported Agriculture Kit payload version"] };
  }

  const u16 = (i) => (b[i] << 8) | b[i + 1];
  const s16 = (i) => {
```

```

    const v = u16(i);
    return v & 0x8000 ? v - 0x10000 : v;
};
const u32 = (i) =>
  ((b[i] * 0x1000000) +
   (b[i + 1] << 16) +
   (b[i + 2] << 8) +
   b[i + 3]) >>> 0;

const environmentTemperature = s16(29) / 100.0;
const environmentHumidity = u16(31) / 100.0;

return {
  data: {
    payload_version: b[0],
    test_sequence: u32(1),
    sensor_uptime_ms: u32(5),
    soil_moisture_percent: u16(9) / 100.0,
    soil_temperature_c: s16(11) / 100.0,
    uv_index: u16(13) / 100.0,
    barometer_pressure_pa: u32(15),
    barometer_temperature_c: s16(19) / 100.0,
    light_veml7700_lux: u32(21) / 100.0,
    light_opt3001_lux: u32(25) / 100.0,
    environment_temperature_c: environmentTemperature,
    environment_humidity_percent: environmentHumidity,
    environment_pressure_pa: u32(33),
    environment_gas_resistance_ohm: u32(37),
    rain_wet: b[41] === 1,
    battery_v: u16(42) / 1000.0,
    sensor_validity_bitmap: u16(44),

    // Existing convenience fields used by the test data path.
    temperature_c: environmentTemperature,
    humidity_percent: environmentHumidity
  }
};
}

```

If Step 8 changed the UV field semantics/name, change the codec consistently before freezing the payload contract.

Step 19 - Compare serial source to decoded output

For at least ten consecutive sequences:

```

EMU-01 source line
  |
  | test_sequence
  | every physical sensor field
  | validity bitmap
  |
  ▼
ChirpStack decoded object

```

Pass only when the same sequence has the expected values after the defined integer scaling/rounding.

Part F - Node-RED and TimescaleDB

Step 20 - Keep all physical fields

Store the complete decoded object in `payload_json` and normalize at least:

```
soil_moisture_percent      %
soil_temperature_c        Cel
uv_index                   1 (only if truly UV index)
barometer_pressure_pa     Pa
barometer_temperature_c   Cel
light_veml17700_lux       lx
light_opt3001_lux         lx
environment_temperature_c Cel
environment_humidity_percent %
environment_pressure_pa   Pa
environment_gas_resistance_ohm ohm
rain_wet                   boolean/state
battery_v                  V
```

Compatibility fields:

```
telemetry.uplinks.temperature_c = environment_temperature_c
telemetry.uplinks.humidity_percent = environment_humidity_percent
telemetry.uplinks.battery_v = battery_v
```

Use quality measured for valid physical measurements. Do not label the sensor data synthetic.

Step 21 - Prove one complete end-to-end record

Pick one `test_sequence` and prove:

```
EMU-01 serial source
  | same seq/value
  ▼
ChirpStack decoder
  | same seq/value
  ▼
Node-RED
  | same seq/value
  ▼
TimescaleDB payload_json
  |
  ▼
selected Fabric evidence
```

At minimum verify:

```
test_sequence
sensor_validity_bitmap
soil value
UV value
barometer value
both light values
RAK1906 values
rain state
```

If any field differs beyond the documented integer scaling/rounding, stop and fix the codec/mapping before counted testing.

Part G - Freeze EMU-01

Step 22 - Create the final baseline

Save:

```
chapter4-results/_device-baseline/  
  EMU-01-hardware.txt  
  EMU-01-firmware.txt  
  EMU-01-sensor-map.txt  
  EMU-01-payload-contract-v2.txt  
  EMU-01-sensor-library-versions.txt  
  chirpstack-physical-sensor-codec.js  
  firmware-source-or-build-hash.txt  
  as923-radio-baseline.txt
```

Do not save keys.

After the baseline is frozen, do not modify the sensor map, payload structure, firmware, BSP, or sensor libraries inside the same counted repetition group.

Part H - SEC-02 after sensor verification

Step 23 - Confirm every B-copy sensor was already tested

Before changing SEC-02 firmware family, require evidence for:

```
SOIL-B  
UV-B  
BARO-B  
LIGHT-VEML-B  
LIGHT-OPT-B  
ENV-B  
RAIN-B
```

This is the hard boundary between sensor verification and security node.

The accepted physical profiles are frozen in [assembly/02b-rak19007-sec02-fixed-profiles.md](#). Before changing firmware family, also require:

```
[ ] Profile A readings and pin-map evidence saved  
[ ] Profile B readings and pin-map evidence saved  
[ ] USB/battery/solar disconnected before the final rebuild  
[ ] Sensor A = EMPTY  
[ ] Sensor B = EMPTY  
[ ] Sensor C = EMPTY  
[ ] Sensor D = EMPTY  
[ ] IO slot = EMPTY
```

```
[ ] RAK4631 Core B remains installed
[ ] LoRa antenna remains attached
```

Do not leave a temporary Agriculture Kit sensor on SEC-02 merely because it was used during verification. The normal security baseline is intentionally stripped to reduce unrelated hardware variables.

Step 24 - Decide the security firmware path

The current test manuals use RUI3/AT-command fixtures for invalid OTAA and raw LoRa P2P security transmissions.

If Core B is still a standard Arduino-BSP RAK4631, convert it to RAK4631-R/RUI3 **only after B-copy verification** using RAKwireless' current official RAK4631-R DFU/conversion procedure.

Do not copy old bootloader/DFU package names blindly from a screenshot or old note. Use the currently published RAK conversion guide and record the exact package/version used.

Physical conversion acceptance - 2026-09-02: PASS. The second RAK4631 plugged in by the operator as SEC-01 is the hardware role this repository has historically documented as SEC-02; do not interpret those labels as two different security nodes unless the project topology is deliberately revised. The board was positively identified on COM13 as the RAK4631 Arduino USB identity VID_239A/PID_8029. The current official RAK two-stage Windows conversion was then executed. The RUI3 bootloader package SHA-256 was F39A6B14964F1CF308908FD4EBAF068316A4827B1E5EFC08F4BF04EAAE33483B; after the 1200-baud transition the DFU port appeared as COM14, VID_239A/PID_002A, and the retry directly on that DFU port reported Device programmed. The final RUI3 package SHA-256 was 7885110C39FA2AF1B207026546A4F33122B81AC7C825939AE3B1C654D7EF155B; Nordic nrfutil 6.1.7 (executable SHA-256 FEE3AEED1843A4BFFC082747CEA60DC072929EB36E67D62C8A3FD7471E0DC92D) programmed it successfully. The application re-enumerated as COM16, VID_1915/PID_521F.

The initial converted RUI3 identity was 3.4.2-rui3_22q1_update.112. During the 2026-09-03 security preflight that old build was found unable to expose the public-LoRa syncword/IQ controls required for a faithful raw LoRaWAN replay/spoof waveform, so SEC was upgraded in place through the official RAK4631-R USB DFU path to **RUI_4.2.4_RAK4631**. The downloaded RAK4631_latest_dfu_package.zip SHA-256 was 738C317DCCDF19B987471AB0620B5AB3CFF6B920A896BC744D2A785EDA9D55CC; Nordic nrfutil 6.1.7 remained SHA-256 FEE3AEED1843A4BFFC082747CEA60DC072929EB36E67D62C8A3FD7471E0DC92D, and DFU returned Device programmed.

RUI3 4.2.4 exposed AT+SYNWORD and AT+IQINVER. SEC was configured temporarily at 923200000 Hz, SF7, 125 kHz, CR 4/5, preamble 8, TX power 14, public LoRaWAN syncword 3444, and normal uplink IQ. One uncounted short P2P rehearsal returned TXP2P DONE; Gateway-01 then received a distinct SF7/923.2 MHz raw frame with gateway uplink ID 1355632844. That uplink ID had no EMU-01 telemetry row, while the nearest legitimate EMU-01 frame used a different uplink ID, proving the RAK5146 raw-RF rehearsal independently. SEC was immediately returned to the safe parked state: NWM=1, BAND=8 / **AS923-1**, NJM=1 (OTAA), CLASS=A, NJS=0, and DEVEUI=0000000000000000. No EMU-01 credential/session material was installed or reused.

After conversion, verify:

```
AT+VER=?
AT+BUILDTIME=?
AT+REPOINFO=?
AT+HWMODEL=?
AT+HWID=?
```

Save sanitized output as:

Step 25 - Configure the wrong-AppKey fixture

Rules:

```
EMU-01 = powered off for this identity-overlap fixture
SEC-02 may use LEGIT_DEV_EUI + LEGIT_JOIN_EUI
SEC-02 MUST use WRONG_APP_KEY
SEC-02 MUST NOT receive LEGIT_APP_KEY
```

Representative RUI3 sequence:

```
AT+NWM=1
AT+NJM=1
AT+BAND=<APPROVED AS923 BAND VALUE>
AT+DEVEUI=<LEGIT_DEV_EUI>
AT+APPEUI=<LEGIT_JOIN_EUI>
AT+APPKEY=<WRONG_APP_KEY>
AT+JOIN=1:0:10:1
AT+NJS=?
```

Do not save the key value in the evidence transcript.

Step 26 - Configure the unregistered-device fixture

Use:

```
DevEUI = <UNREGISTERED_DEV_EUI>
JoinEUI = dedicated test JoinEUI
AppKey = dedicated test fixture key
```

Confirm the test DevEUI does **not** exist in ChirpStack before counting an unregistered-device attempt.

Step 27 - Prepare raw LoRa P2P mode

Representative RUI3 controls:

```
AT+NWM=0
AT+P2P=<freq>:<sf>:<bw>:<cr>:<preamble>:<power>
AT+SYNCWORD=<captured/required LoRaWAN-compatible value>
AT+IQINVER=<uplink-compatible value>
AT+PCRYPT=0
AT+PRECV=0
AT+PSEND=<RAW_PHYPAYLOAD_HEX>
```

Do not guess RF settings. Copy frequency/data-rate-related parameters from the actual legitimate captured packet/gateway evidence used by the authorized test.

Step 28 - Prove SEC-02 RF reception before counted replay/spoof tests

```
SEC-02 raw transmit
  |
  v
RAK5146 receives RF
  |
  v
gateway evidence saved
```

If RAK5146 does not receive the rehearsal frame, replay/spoofing tests are blocked even if EMU-01 LoRaWAN works normally.

Final setup acceptance before sensor preflight

Do not enter counted execution from this manual. First satisfy this setup gate, then run the dedicated sensor preflight:

```
[ ] both RAK4631 cores have permanent identities/labels
[ ] all A/B direct sensors were physically verified
[ ] EMU-01 keeps complete A-set installed
[ ] final Arduino firmware uploaded to EMU-01
[ ] validity bitmap = 0x007F during healthy pre-run cycles
[ ] payload v2 frozen at 46 bytes
[ ] production scheduler proven: 60-second local sampling, DevEUI-staggered 5-minute nominal unconfirmed
uplinks with ±15-second jitter, and rate-limited randomized rain-event uplinks
[ ] source log matches ChirpStack decoder for ten sequences
[ ] EMU-01 OTAA join succeeds
[ ] one physical-sensor record reaches TimescaleDB
[ ] one selected record completes the Fabric evidence path
[ ] firmware/BSP/library versions are recorded
[ ] no legitimate keys are present in Git/evidence/SEC-02
[ ] SEC-02 B-copy verification completed before security conversion
[ ] SEC-02 wrong-AppKey fixture works
[ ] SEC-02 unregistered-device fixture works
[ ] SEC-02 raw LoRa frame is received by RAK5146
```

When this setup gate passes, continue to [preflight/00-README.md](#). The preflight re-tests the **frozen final configuration as one system** and must produce `SENSOR_PREFLIGHT_STATUS=GO` before Execution 01 begins.

Official references

- RAK4631 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak4631/quickstart/>
- RAK4631 overview: <https://docs.rakwireless.com/product-categories/wisblock/rak4631/overview/>
- WisBlock quick start: <https://docs.rakwireless.com/product-categories/wisblock/quickstart/>
- RAK4631-R conversion / DFU: <https://docs.rakwireless.com/product-categories/wisblock/rak4631-r/dfu/>
- RUI3 AT commands: <https://docs.rakwireless.com/product-categories/software-apis-and-libraries/rui3/at-command-manual/>
- Agriculture Kit: <https://docs.rakwireless.com/product-categories/wisblock/kit7-agriculture/overview/>

Supporting runbook: 04-replay-spoofing.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Execution 4. LoRaWAN Replay and Spoofing

Reference procedure only. Executable operator commands are published only in `chapters/lorawan_research_test_manual_final.md` after live qualification. The fenced snippets below are preserved as implementation/reference text and must not be copied as current commands.

Perform this only on the isolated/authorized LoRaWAN test environment.

This is an independent counted LoRaWAN experiment and may start under `LORAWAN_TRACK_STATUS=GO` even while the full-stack status remains NO-GO solely because external Fabric is unavailable. Record `RUN_SCOPE=LORAWAN_ONLY`. A rejected replay/invalid-MIC frame is decided by ChirpStack before the application path, so a successful external Fabric commit is not part of the attack-decision prerequisite.

Required design

```
Replay:
  10 new legitimate control uplinks
  10 received replays of older accepted uplinks

Spoofing:
  10 genuine control uplinks
  10 received forged uplinks with invalid MIC

Total counted attempts: 40
```

An attack transmission counts only when the gateway actually receives the RF frame.

Equipment

```
RAK4631 EMU-01 registered full physical-sensor OTAA node
RAK5146 gateway
RAK4631 SEC-02 raw-RF security test node
ChirpStack gateway/frame logging
server MQTT/application/database/Fabric logs
```

Keep normal frame-counter validation enabled. Do not disable security checks to make the attack easier.

What this test proves

Replay and spoofing are separate conditions:

```
Replay -> exact old legitimate LoRaWAN bytes are transmitted again after the legitimate counter has advanced.
```

Spoofing -> a frame looks like the legitimate device at the address level but cannot authenticate because its MIC is invalid.

The gateway is not the security decision point. It may receive both frames. The security result is whether ChirpStack turns them into accepted application uplinks.

Before the first counted attempt

1. Run the Execution 01 preflight.
2. Confirm EMU-01 is joined and stable.
3. Confirm SEC-02 contains no legitimate AppKey/session keys.
4. Perform one **uncounted** raw-RF rehearsal from SEC-02 and prove RAK5146 reception.
5. Confirm gateway evidence exposes enough information to save the raw PHYPayload and RF parameters from a legitimate uplink.
6. Create `replay-spoofing/trial-results.csv`.
7. Keep the gateway and ChirpStack log captures running for the complete 40-attempt group.

If step 4 or 5 fails, the counted test is blocked.

Part A - Replay test

Step A1 - Create a replay trial folder and capture one legitimate uplink

For each replay trial create a separate folder, for example:

```
replay-R01/  
replay-R02/  
...  
replay-R10/
```

For each trial:

1. Let EMU-01 send a new legitimate physical-sensor uplink and record its deterministic `test_sequence`.
2. Confirm the gateway receives it.
3. Confirm ChirpStack accepts it.
4. Confirm it reaches Node-RED and TimescaleDB.
5. When selected for Fabric, confirm the normal outbox/Fabric result.
6. Record:

```
raw PHYPayload  
DevAddr  
frame counter  
frequency  
data rate / spreading factor / bandwidth  
reception time  
gateway EUI  
source event key
```

Save the capture only in the protected test-result directory.

This legitimate message is the control for that replay trial.

The workstation helper `test/automation/research-recorder/sec02_replay.py` may build this fixture directly from the immutable Gateway-01 journal plus the correlated gateway MQTT evidence. The physical board labelled SEC-01 by the operator is the same security-node role historically named SEC-02 in this repository. The helper never provisions EMU-01 keys; it handles only already-captured ciphertext PHYPayload bytes and observed RF parameters.

For a fresh uncounted rehearsal or a supervised counted trial, write the fixture under that run's protected `chapter4-results/` directory. The helper defaults to requiring the selected accepted EMU-01 packet to be at least three FCnt values behind the latest accepted EMU-01 FCnt and refuses capture otherwise:

```
python .\test\automation\research-recorder\sec02_replay.py capture `
--output <RUN-DIR>\replay-R01\fixture.json `
--minimum-fcnt-age 3

python .\test\automation\research-recorder\sec02_replay.py send `
--fixture <RUN-DIR>\replay-R01\fixture.json `
--dry-run
```

Only after the dry-run output is reviewed and the formal recorder is already retaining EMU-01, SEC, Gateway-01 and cloud evidence should the same send command be run without `--dry-run`. The helper returns the RUI3 security node to its parked AS923 OTAA state after the transmission attempt.

Step A2 - Advance the legitimate frame counter

Allow EMU-01 to transmit at least three additional uplinks. Confirm both `test_sequence` and the LoRaWAN frame counter advance beyond the captured value.

This makes the captured frame clearly old.

Step A3 - Retransmit the captured frame

Using SEC-02 in the raw LoRa P2P transmit mode prepared in [Sensor Preparation](#), retransmit the **same captured LoRaWAN PHYPayload bytes** using the captured/legal RF parameters required for the gateway to receive it.

Transmit in a quiet gap between EMU-01's scheduled 15-second **counted-test-profile** uplinks when practical so an RF collision does not make the trial ambiguous. Do not change the cadence profile, stop frame-counter validation, or alter ChirpStack settings.

Do not rebuild, decrypt, re-encrypt, or modify the replay PHYPayload. Do not alter ChirpStack frame-counter settings. SEC-02 must not contain EMU-01's legitimate AppKey or session keys.

Step A4 - Prove RF reception

Check the gateway/gateway-frame evidence.

If the RAK5146 did not receive the replay transmission, mark the attempt **invalid and repeat it**. A missed RF transmission is not replay rejection.

Step A5 - Prove downstream rejection

For a valid received replay, verify all of these:

```
gateway received replay = yes
new ChirpStack application uplink = no
MQTT/Node-RED accepted event = no
new TimescaleDB row from replay = no
new Fabric outbox/ledger work attributable to replay = no; when the external ledger is unavailable, prove the
```

```
stronger upstream fact that no accepted application/DB/outbox event exists and record external-ledger
verification as NOT_REQUIRED_FOR_REJECTED_FRAME
```

Record the ChirpStack rejection/retransmission evidence and decision time when timestamps permit.

Repeat until there are **10 legitimate controls and 10 received replay attempts**.

Part B - Spoofing / invalid-MIC test

Step B1 - Generate a genuine control uplink

For each trial, let EMU-01 transmit its normal physical-sensor payload and retain the deterministic test_sequence as part of the control evidence.

Confirm:

```
gateway receives
ChirpStack authenticates
application event exists
TimescaleDB record exists
```

Repeat for 10 genuine controls.

Step B2 - Prepare a forged frame

For every spoofing trial, retain both values/files:

```
original captured PHYPayload
modified spoofing PHYPayload
```

The modified fixture must preserve the controlled legitimate device address while changing protected bytes or the MIC such that the original legitimate MIC is no longer valid. Do not recompute a valid MIC and do not provision SEC-02 with the legitimate session key.

Before transmission, record which byte/field was intentionally changed and verify the final MIC was not replaced with a newly valid MIC.

Use the controlled spoofing fixture plus SEC-02 raw transmit mode to create/transmit a test uplink that:

```
uses the legitimate device address for the controlled test device
contains controlled modified MACPayload / encrypted application bytes representing forged content
uses a MIC that was not recomputed with the legitimate session key
uses RF parameters the gateway can receive
```

SEC-02 must not contain EMU-01's legitimate network session key. The purpose is to prove that a frame that looks like the device at the address level still fails LoRaWAN authentication. Because an outsider without the session keys cannot encrypt a chosen meaningful application value correctly, do not claim that SEC-02 produced a valid semantic temperature/soil reading; claim only the controlled address-impersonation / modified-frame condition that ChirpStack must reject.

Step B3 - Transmit and prove gateway reception

Transmit the forged frame in the authorized test environment.

If the gateway does not record reception, do not count the attempt. Correct RF parameters and repeat.

Step B4 - Prove ChirpStack rejection

For each received forged frame verify:

```
gateway reception = yes
ChirpStack accepted application uplink = no
MIC/authentication rejection evidence = yes
MQTT/Node-RED accepted event = no
TimescaleDB row = no
Fabric outbox/ledger work attributable to forged frame = no; when the external ledger is unavailable, prove
no accepted application/DB/outbox event exists and record external-ledger verification as
NOT_REQUIRED_FOR_REJECTED_FRAME
```

Repeat until **10 received forged attempts** are recorded.

Database check

For every attack trial, use its time range, DevEUI, frame-counter/source identity, or controlled test marker to confirm no corresponding accepted row was inserted.

Example inspection:

```
docker compose exec telemetry-db \
  psql -U telemetry_admin -d lorawan_telemetry \
  -c "SELECT event_key,time,dev_eui,f_cnt FROM telemetry.uplinks WHERE dev_eui='<TEST_DEV_EUI>' ORDER BY time
DESC LIMIT 30;"
```

Also inspect recent outbox rows:

```
docker compose exec telemetry-db \
  psql -U telemetry_admin -d lorawan_telemetry \
  -c "SELECT event_key,status,fabric_tx_id,created_at FROM telemetry.fabric_outbox ORDER BY created_at DESC
LIMIT 30;"
```

Required CSV fields

```
test
condition
trial
control_test_sequence
gateway_received
chirpstack_accepted
chirpstack_rejected
reached_mqtt_node_red
database_record_created
fabric_transaction_created
false_acceptance
false_rejection
decision_time_seconds
evidence_reference
```

After each attack trial

Before moving to the next trial:

1. write the CSV row immediately;
2. save the matching gateway reception reference;

3. save the matching ChirpStack decision reference;
4. query the database/outbox for accidental propagation;
5. classify the attempt PASS, FAIL, or INVALID;
6. if INVALID, keep the evidence and rerun under a new attempt ID;
7. confirm a normal EMU-01 uplink still succeeds before continuing if the failure might indicate a broken testbed rather than attack rejection.

Pass condition

```
40 counted attempts exactly
all legitimate controls accepted
all 10 received replay frames rejected before application processing
all 10 received invalid-MIC frames rejected before application processing
zero attack-created database rows
zero attack-created Fabric outbox jobs; zero attack-created Fabric ledger transactions when an external
ledger is active. Under the current LoRaWAN-only scope, absence of any accepted application event and outbox
job is sufficient downstream-propagation proof.
```

Continue to [Execution 05 - Data Integrity](#).

Supporting runbook: README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Automated Research Recorder

This is the preferred evidence-capture path for counted dissertation tests on the **commissioned** Gateway-01 + LTE + three-node cloud system.

Evidence authority

The testing evidence is authoritative for observed results. The current methodology authority for what must be tested and measured is [../..ZACARIAS-CHAPTER3-TEST-METRICS-SOURCE-OF-TRUTH.md](#), derived from the new chapters/Zacarias_Chapter3.pdf. Use it for research questions, experimental conditions, counts, durations, measurements, analysis rules, and stated acceptance/security criteria. Do not treat architecture, topology, technology placement, host roles, status statements, or expected outcomes in that PDF—or older dissertation drafts—as current infrastructure truth.

Use this order when sources disagree:

1. sealed evidence captured for the specific formal trial under chapter4-results/ is the authority for that trial and its research classification;
2. fresh read-only live observations are the authority for current operational state at the time they were observed;
3. current deployment/configuration material is the reproducibility reference;
4. chapter PDFs, old DOCX files and historical prose are requirements/provenance context only.

A healthy live service does **not** prove that a research trial passed. The recorder deliberately captures evidence first and leaves classification to the evidence from that trial. The implementation must follow the live commissioned system rather than recreating obsolete architecture described in older drafts.

What is automated

One recorder run automatically retains:

- EMU-01 USB serial with host UTC timestamps and automatic COM-port re-enumeration recovery;
- optional SEC USB serial capture;
- Gateway-01 live logread stream;
- Gateway-01 CPU, memory, load and network counters;
- Gateway-01 LTE/QMI, route and production MQTT-socket snapshots;
- host CPU, memory, load and network counters on ulc-01, ulc-02, ulc-03;
- per-container CPU/memory series on all three cloud nodes;
- exact-window ChirpStack, Node-RED, Mosquitto and kernel logs;

- authoritative telemetry.uplinks, telemetry.measurements, and telemetry.fabric_outbox CSV exports from the current Patroni leader;
- RSSI, SNR, gateway frequency, gateway uplink ID, FCnt and test_sequence statistics;
- byte-exact EMU-01 source-to-database **delivery correlation** whose source timestamps can be clock-aligned into the authoritative DB export window; this is not the formal Chapter 3 PDR;
- formal Chapter 3 PDR is reported only when the numerator can be derived as unique legitimate uplinks accepted by ChirpStack over the same eligible source-attempt window;
- application-event-to-database timing (received_at - time) plus independently calibrated **sensor-transmission-to-database-storage latency** from matched SENSOR_TX source timestamps when recorder-v9 start/end clock calibration passes;
- a read-only aggregate verifier/checkpoint snapshot at run start and run end (metadata/evidence-status-start.txt and metadata/evidence-status-end.txt);
- SHA-256 for every completed evidence file.

Raw files are made read-only after finalization. Calculated files live under derived/; never edit raw evidence to improve a result.

Live testing monitor

research_cockpit.py provides the read-only testing monitor at <http://127.0.0.1:8765/>. It is an observation aid for running experiments, not a second result engine.

The cockpit combines the existing live sensor/resource view with testing_monitor.py. Each cloud node uses one locked-down SSH monitor-stream session so monitoring does not create a burst of competing SSH probes during a trial. Resource records remain approximately one-second observations; slower service/database/evidence observations are multiplexed through the same session. The server wrapper discovers the live Docker/service set instead of relying on chapter-era service lists.

The monitor shows:

- current CPU utilization as percent of 100%, RAM used/total, and network receive/send rates in Mbps with explicit capacity context where a defensible capacity is known;
- received sensor packets by sender, DevEUI, gateway, AS923 radio frequency, RSSI in dBm, SNR in dB, FCnt/test sequence and decoded sensor values;
- packet-level RF-to-application correlation from Gateway-01 reception through gateway MQTT evidence, ChirpStack/application acceptance, TimescaleDB, evidence verification, Fabric outbox and Fabric transaction fields when present;
- duplicate/replay indicators that distinguish an exact PHYPayload received again under a new gateway uplink ID from ordinary MQTT redelivery, plus FCnt/test-sequence repeats, gaps and regressions;
- discovered containers and selected system services with node, role/state and live detail;
- current Patroni leader/replica observations;
- evidence collector/verifier and Fabric adapter readiness, including the deliberately fenced Fabric standby when observed;
- leader-only database counters for LoRaWAN uplinks, decoded measurements, gateway MQTT evidence, evidence segments, checkpoints, verifications and Fabric outbox rows;
- recent warning/error observations and the formal test inventory from chapter4-results/.

Operational observation history is stored locally in %LOCALAPPDATA%\LoRaWAN\testing-monitor.sqlite3 using SQLite/WAL. Packet observations are stored separately in %LOCALAPPDATA%\LoRaWAN\packet-monitor.sqlite3. Service snapshots are persisted at a bounded cadence (currently 60 seconds), packet correlation is refreshed every five seconds, the UI exposes recent history, and observations are retained for seven days. These local

histories are useful for diagnosing service and packet behavior; they are **not** substituted for the sealed raw/metadata/derived files of a formal trial.

The packet monitor deliberately uses three different duplicate concepts. An exact `phy_payload_sha256` observed with a **new gateway uplink ID** is a strong RF replay/duplicate candidate. Re-observation of the same gateway uplink ID is transport/capture redelivery and is not counted as another RF packet. Duplicate application identity, `FCnt` or `test_sequence` is a downstream/session anomaly and is reported separately. This distinction prevents QoS 1 MQTT behavior from being misreported as a LoRaWAN replay.

Clock and timing gate

Counted runs fail closed unless the source-clock relationship to authoritative cloud UTC is **measured and stable**. `serial_capture.py` timestamps each EMU-01 `SENSOR_TX` line with the workstation clock, while the authoritative run window and database evidence use cloud/server UTC. Recorder v9 therefore does not assume that the workstation wall clock is correct: it independently calibrates the workstation against `ulc-01` immediately before the formal window, repeats the calibration after collectors stop, and invalidates finalization if the start/end mapping drifts by more than **0.100 s**. The calibrated mapping is retained in `metadata/run-meta.json` and is used to derive the Chapter 3 sensor-transmission-to-database-storage latency.

The recorder preflight still takes five midpoint-corrected `utc-now` samples from each of `ulc-01`, `ulc-02`, and `ulc-03`. The three cloud clocks must agree within **0.250 s**, Gateway-01 must remain within **±1.500 s** of the cloud median, and the workstation/cloud offset must be measurable within the bounded calibration range (**±300 s**). Absolute workstation offset is recorded rather than silently treated as zero. A large offset is acceptable only when start/end calibration remains stable; cloud disagreement, gateway disagreement, missing calibration, excessive drift, or negative calibrated sensor-to-database latency invalidates timing evidence.

Historical staging diagnosis found that the domain workstation could inherit a large absolute offset from its PDC even while the cloud and gateway clocks remained mutually coherent. Recorder v9 records that offset explicitly and proves its stability at both run boundaries, so a large but stable workstation wall-clock offset no longer blocks a run or contaminates source timing. The Active Directory time source should still be repaired through the normal PDC hierarchy for operational correctness; do not point a domain member directly at public NTP merely to satisfy research tooling.

Gateway-01 remains independently gated against the cloud median because gateway radio/evidence timestamps are part of the cross-system observation boundary. Its limit remains **±1.500 s**. If gateway/cloud agreement or cloud-node spread exceeds its limit, the recorder still fails closed. The restricted research-recorder key remains read-only and must not be weakened to repair time service configuration.

The cockpit's **Overall staging readiness** panel combines this clock gate with live cloud streams, Gateway management visibility, Patroni role topology, evidence readiness, leader database counters, telemetry-read availability, and the absence of an already-active formal recorder. This panel is an operational preflight aid only; the recorder CLI remains the final start gate and additionally checks required local USB/serial devices.

The server side remains read-only. `DBMETRIC` records are emitted only by the node that is observed as the PostgreSQL leader, avoiding duplicate HA counts. If a gateway management observation becomes unavailable, the cockpit shows **Management observation unavailable** and **Not observed** for unavailable gateway metrics; this means the management path could not be observed and does not prove the gateway or LoRaWAN radio path is offline. Sensor telemetry has separate semantics: a successful database read with zero rows is `idle` / **No fresh packets**; a failed live query is **Telemetry observation unavailable**; a transient read failure with a previous valid snapshot retains that snapshot as `stale`; only a read failure with no usable snapshot is an error. The clean fresh-test baseline is therefore expected to show **No fresh packets** until real new AS923 uplinks arrive. Telemetry queries use bounded five-second reads and a 30-second cache horizon, and their SSH subprocesses explicitly use a closed stdin (`DEVNULL`) so a detached/managed cockpit worker cannot block waiting on inherited console input. These rules prevent a slow polling cycle or unavailable management path from being mislabeled as device failure.

Recorder v4 adds a **foreground supervisor**. `run` is now the preferred Chapter 4 entry point: it starts the same serial/remote collectors as `start`, performs a bounded collector/evidence health gate every 15 seconds by default, records those observations in `raw/control-plane-health.csv`, and invokes the normal `export/summarize/hash` finalizer automatically on duration completion or `Ctrl+C`. This prevents an unnoticed workstation-session loss from leaving stale active state and silently invalidating later trials. Each supervisor row records the server-authoritative UTC timestamp, the workstation midpoint timestamp around that clock probe, the measured server-minus-workstation offset, and probe round-trip time. This makes workstation clock skew explicit instead of silently mixing clock domains. Experiment timing still uses server-authoritative timestamps and source/database evidence as described below.

Recorder v5 hardens that supervisor for real multi-minute counted work. A dead collector process remains an immediate terminal failure because evidence is actually being lost. Short-lived remote-readiness, USB/re-enumeration, database-role, or clock-probe failures are instead retained as `WARN` observations and must persist for three consecutive health gates before the supervisor aborts; a later clean gate resets the transient-failure count. Per-node SSH/readiness timeouts are converted into explicit health details rather than uncaught exceptions. If the server-authoritative clock probe is temporarily unavailable, the CSV leaves the authoritative timestamp and skew fields blank, records the condition as a warning, and never substitutes workstation time as if it were server time. `status`, `stop`, and `recover` also tolerate a failed final health probe so preservation/finalization can still complete. This policy was added after the 2026-09-04 uncounted counted-profile reload capture showed that a single transient readiness timeout could terminate the foreground supervisor while all child collectors and the actual LoRaWAN path remained healthy.

Workstation dependency

The recorder is Python-native because this workstation is governed by a domain PowerShell `MachinePolicy=Restricted`. Do not weaken that policy.

Python dependency is pinned in `requirements.txt`:

```
pyserial==3.5
```

Install/reproduce with:

```
python -m pip install --user -r .\lorawan-network-server-gateway\test\automation\research-recorder\requirements.txt
```

The older `.ps1` prototypes in this directory are not the operator entry point on this workstation.

Locked-down SSH boundary

The recorder uses a dedicated key outside the repository:

```
%USERPROFILE%\ssh\id_ed25519_research_recorder
```

The private key is never stored in the project. Its public key is installed on the three cloud nodes and Gateway-01 with a forced-command wrapper. The key can invoke only the `read/capture` verbs implemented under `remote/`; an arbitrary command such as `uname -a` is rejected.

Never replace this with an embedded SSH password or copy an administrator private key into the repository.

Forced-command monitor boundary - current state 2026-09-18

Fresh restricted-key version checks on ulc-01, ulc-02, and ulc-03 all return research-recorder-server-v7, matching the current repository wrapper. Do not reuse the historical v5 wrapper hash as a v7 integrity claim; verify the deployed file hash separately whenever byte-for-byte wrapper attestation is required. Gateway-01 remains on its separately versioned restricted gateway wrapper. The dedicated recorder key remains forced to reviewed verbs; arbitrary shell commands are not permitted.

Server wrapper v7 retains the bounded recorder/readiness verbs, monitor-stream <seconds>, and the fixed read-only db-export packetflow <startUTC> <endUTC> <gatewayEUI> projection used by the live packet correlator. The monitor stream multiplexes one-second host-resource records with slower service, readiness, database-role, database-counter and bounded recent-error observations. Optional service/error side probes are deliberately non-fatal, so an empty error scan or one failed auxiliary observation cannot terminate the resource stream and silently leave the cockpit stale. The packetflow projection joins already-authoritative gateway MQTT evidence to accepted telemetry, evidence-verification status, Fabric outbox state and Fabric transaction fields. It accepts no arbitrary SQL, table name, column name or filter expression from the caller.

Gateway wrapper v2 retains snapshot, resource and logstream and adds bounded radio-recent <count> and journal-recent <count> reads. radio-recent exposes only recent RAK5146 frame-reception log lines. journal-recent exposes retained immutable gateway journal records needed to correlate exact PHYPayload bytes for authorized replay testing. Neither command changes radio, network or journal state.

The version verbs and the fixed packetflow/radio reads were verified through the same restricted recorder identity after deployment. Keep the repository wrappers and deployed copies converged; do not bypass or weaken the forced-command boundary to troubleshoot monitoring.

Read-only evidence-status boundary

The bounded evidence-status capability remains part of the current wrapper. It executes fixed SELECT-only queries against the commissioned evidence status views and returns aggregate verification/checkpoint state only. It exposes no payload bytes, evidence hashes, database credentials, arbitrary SQL, or arbitrary shell access.

The older UNAVAILABLE_WRAPPER_UPGRADE_REQUIRED compatibility behavior is historical only. Current cloud nodes run wrapper v7, so new formal runs are expected to capture the fixed evidence-status output when the underlying evidence read path is healthy. If a current run receives command not allowed or an unavailable-wrapper marker from a cloud node, treat that as a deployment/version regression and investigate it rather than accepting it as the present baseline.

Journal-read prerequisite

opsadmin must be a member of systemd-journal on ulc-01, ulc-02, and ulc-03. This is required so the forced-command recorder identity can retain Mosquitto and kernel-journal evidence without using sudo or an unrestricted administrator shell. The membership was applied and verified on all three cloud nodes on 2026-09-03.

Accepted recorder smoke - 2026-09-03

The first uncounted smoke, recorder-smoke-20260903, proved the core start/capture/stop/export/summarize/hash path but exposed the missing journal-read permission above; preserve that run as diagnostic evidence. After correcting journal access, recorder-smoke-journal-20260903 completed cleanly with all eight live collectors healthy, recorder stop exit 0, status RECORDED_UNCLASSIFIED, no stop-error file, 65 SHA-256 manifest entries independently verified, and 39 raw evidence files sealed read-only. The marker path, TimescaleDB exports, exact-window service logs, kernel logs, statistics, and active-state cleanup were also verified. Static/live recorder validation then passed.

Both smoke runs intentionally used --skip-serial. Counted LoRaWAN runs must pass preflight --require-emu --require-sec and must not skip EMU-01 source capture when the PDR denominator depends on SENSOR_TX

attempts.

Accepted supervised-recorder smoke - 2026-09-04

Recorder v4 was commissioned with two bounded, uncounted foreground run `--skip-serial` smokes against the live Gateway-01 plus three-node cloud. The first run, `collector-supervised-smoke-20260904-0456z`, proved collector supervision and clean automatic finalization but deliberately remains diagnostic evidence because it exposed that its first implementation timestamped supervisor health with workstation UTC while start/end boundaries used server-authoritative UTC. The workstation was materially behind the cloud clock, so those control-plane rows could appear before the run start. The same run also showed that uncounted metadata must not claim the frozen counted EMU profile when serial capture is skipped.

Both provenance defects were corrected before acceptance. `control-plane-health.csv` now records server-authoritative UTC, the bracketing workstation midpoint, measured server-minus-workstation offset, and clock-probe round-trip time. `--skip-serial` metadata now records `serial_capture_enabled=false`, `emu_profile=NOT_CAPTURED_UNCOUNTED`, and leaves counted source/HEX hashes blank rather than implying the counted firmware was observed.

The corrected run `chapter4-results/smoke/collector-supervised-clockfix-20260904` is the accepted v4 supervisor smoke. It exited 0, finalized as `RECORDED_UNCLASSIFIED`, produced 76 manifest entries with zero missing files and zero SHA-256/size mismatches, left no `_recorder-active.json`, `stop-errors.txt`, or `supervisor-errors.txt`, and recorded three monotonically increasing `PASS` supervisor rows inside the authoritative start/end window. The clock probes measured the server about 104.65-104.90 s ahead of the workstation with roughly 1.06-1.22 s round-trip time; this skew is now explicit evidence rather than a hidden timing assumption. A subsequent live preflight `--require-emu --require-sec` also passed with EMU-01 on COM11, SEC on COM16, `u1c-01` leader, `u1c-02/u1c-03` replicas, and all required evidence roles ready.

Counted-profile post-flash sanity - 2026-09-03

The uncounted readiness run `counted-profile-postflash-sanity-20260903` exercised the frozen 15-second EMU-01 image with real USB source capture and the commissioned Gateway-01/LTE/cloud path. The sealed recorder-v1 bundle contains 189 healthy source attempts (`send_status=0`, sequences 7..195), 192 unique database event keys, 2,496 normalized measurements (exactly 13 per event), and 192 Fabric-outbox rows left pending because the external Fabric worker remains intentionally disabled.

A naïve comparison of 192 DB rows against 189 captured source attempts produced an impossible 101.587% value in the sealed v1 derived summary. That number is **not a valid PDR and must not be cited**. Byte-for-byte reconstruction of each 46-byte payload proved 185 exact source/DB matches for sequences 7..191. Those matches establish a stable DB-minus-workstation timestamp offset of about +103.24 seconds. After applying that measured offset to define the source attempts actually covered by the DB export window, all 185 eligible source attempts have exact DB matches: aligned PDR = 100.0%. DB-only rows are the six current-session frames emitted before serial capture plus one prior-session sequence-8 frame admitted by the skewed wall-clock start boundary; source-only sequences 192..195 project beyond the DB export end.

Recorder version 2 therefore no longer calculates PDR as `DB-row-count/source-attempt-count`. It reconstructs the payload-v2 bytes from every `SENSOR_TX` line, correlates them exactly to `raw_data`, estimates the workstation/cloud clock offset from those exact matches, and limits the denominator to source attempts whose projected DB timestamps fall inside the actual export window. If that aligned observation window cannot be established, PDR is withheld rather than guessed. The sealed v1 bundle is intentionally left unchanged; corrected interpretation is recorded in the preflight analysis outside the sealed run.

The same sanity run exposed a Windows-specific shutdown defect: `taskkill /T /F` could hang while traversing the process tree. Direct `taskkill /PID <pid> /F` was proven against the live recorder-owned processes and a disposable child. Both Python and legacy PowerShell recorder paths now omit `/T`; the Python path also bounds termination to five seconds.

Result shape

```
chapter4-results/<group>/<run-id>/
raw/
  emu-01-source.log
  sec-source.log           # when requested
  gateway.log
  gateway-resource.csv
  ulc01-host-resource.csv
  ulc01-docker-resource.csv
  ulc02-host-resource.csv
  ulc02-docker-resource.csv
  ulc03-host-resource.csv
  ulc03-docker-resource.csv
  uplinks.csv
  measurements.csv
  fabric-outbox.csv
  *-chirpstack.log
  *-mosquitto.log
  *-node-red.log
  *-kernel.log
  markers.csv
metadata/
  run-meta.json
  start-utc.txt
  end-utc.txt
  *-snapshot-start.txt
  *-snapshot-end.txt
  database-roles-end.json
  SHA256SUMS.csv
derived/
  run-summary.json
  run-summary.csv
  trial-results.csv
  run-status.txt
```

PASS, FAIL, INVALID, and BLOCKED remain research classifications. The recorder deliberately ends a clean capture as RECORDED_UNCLASSIFIED; evidence collection must not invent the experiment outcome.

Usage

From the repository root:

```
$R = './lorawan-network-server-gateway/test/automation/research-recorder/research_recorder.py'

# Remote collectors + Patroni topology; add both flags before LoRaWAN security testing.
python $R preflight --require-emu --require-sec

# Preferred counted-run mode. Keep this terminal open; Ctrl+C finalizes safely.
# Use a second terminal for mark/trial/serial-control commands while this runs.
python $R run `
  --group authentication `
  --run-id auth-lorawan-A1-01 `
  --condition 'registered device with correct OTAA credentials' `
  --expected 'allow; JoinAccept and accepted uplink' `
  --scope lorawan `
  --capture-sec

# Timestamp a physical/software event without stopping capture.
python $R mark --message 'A1-01 reset issued'

# Read the current aggregate verifier/checkpoint state without starting a run.
# This is intentionally strict: on pre-v3 live wrappers it reports that the server-side upgrade is required.
python $R evidence-status

# Optional explicit active-recorder view; the foreground supervisor already gates health every 15 seconds.
python $R status
```

```

# When a counted OTAA trial requires reloading the exact frozen EMU image,
# keep the recorder and all remote collectors alive while intentionally yielding COM11.
# The serial log records RECORDER_SERIAL_PAUSED / RECORDER_SERIAL_RESUMED and then
# automatically reacquires the board after DFU re-enumeration.
python $R serial-pause --device emu
# reload the already-frozen counted-test ZIP; do not rebuild here
python $R serial-resume --device emu

# The same mechanism may yield SEC during A2/A3 fixture commands without stopping
# the rest of the counted evidence capture:
# python $R serial-pause --device sec
# ... issue only the documented SEC test-fixture commands ...
# python $R serial-resume --device sec

# Record the trial classification after its evidence is inspected.
python $R trial `
  --trial-id A1-01 `
  --condition 'correct OTAA credentials' `
  --expected allow `
  --actual allow `
  --status PASS `
  --notes 'JoinRequest received; JoinAccept; application uplink stored'

# Normally press Ctrl+C in the foreground `run` terminal when complete.
# It automatically exports DB/log evidence, calculates statistics, hashes files and seals raw evidence read-only.

# Recovery only: if a detached or interrupted run left _recorder-active.json,
# inspect it first, then finalize and preserve it rather than deleting it.
python $R status
python $R recover

```

For a bounded **uncounted** collector smoke test, run `--skip-serial --duration-seconds 60` is appropriate. In that mode `run-meta.json` explicitly marks serial capture disabled and the EMU profile as `NOT_CAPTURED_UNCOUNTED`; it does not claim that the frozen counted-test image was observed. Counted runs normally omit `--duration-seconds` and run until operator Ctrl+C.

Use `--interval 1` for flooding and normally `--interval 5` for baseline/resilience/resource observations. The default supervisor health interval is 15 seconds; do not make it aggressively shorter because each gate also checks the replicated evidence path.

`--skip-serial` exists only for clearly uncounted recorder smoke tests. Do not use it for a counted run whose delivery/PDR denominator depends on EMU-01 `SENSOR_TX` attempts.

Statistical interpretation

The recorder derives only metrics supported by its actual evidence. Formal Chapter 3 PDR and source-to-database delivery are deliberately separate:

```

formal Chapter 3 PDR = unique accepted EMU-01 ChirpStack deduplication IDs across both HA ChirpStack logs /
captured scheduled SENSOR_TX attempts x 100
database delivery    = exact DB payload matches for eligible source attempts / eligible source attempts x 100

```

For formal PDR, `summarize-run.py` strips ANSI log formatting, unions accepted `deduplication_ids` from `ulc01-chirpstack.log` and `ulc02-chirpstack.log`, associates them with the run's `dev_eui` from `run-meta.json`, and divides that unique accepted count by every captured scheduled `SENSOR_TX` attempt in the same run window. A scheduled source-side send failure remains in the denominator. The percentage is withheld when required logs/device identity/source attempts are missing or when accepted uplinks exceed the source-attempt boundary. This is the Chapter 3 LoRaWAN/ChirpStack metric boundary.

For database delivery, source and database evidence are correlated by reconstructing the exact 46-byte payload-v2 bytes from each `SENSOR_TX` line and matching those bytes to `telemetry.uplinks.raw_data`. Exact matches also provide the measured DB-minus-source clock offset. A source-side attempt remains in the delivery-correlation denominator only when the recorder can prove that attempt lies inside the DB observation window. Boundary attempts outside that comparable window are reported as unmatched correlation evidence, not silently counted as packet loss.

Never copy database delivery into a Chapter IV PDR column merely because both percentages may happen to be equal in a healthy run. An MQTT redelivery is not a second LoRaWAN delivery, and a second LoRaWAN session reusing a sequence/FCnt value is not collapsed merely because the counter repeats.

For replay/spoofing, only attack transmissions whose RAK5146 reception is proven belong in the counted attack denominator. `sec02_replay.py capture --test-sequence <N>` may now bind a replay/spoofing fixture to the exact accepted EMU-01 `test_sequence` designated as that trial's legitimate control; the helper still requires the configured minimum FCnt age and independently correlates the selected packet to Gateway-01 journal, packetflow, and RAK5146 reception evidence before it will create the fixture. This prevents a nearby historical uplink from being silently substituted for the intended control.

`application_event_to_db_ms` is `telemetry.uplinks.received_at - telemetry.uplinks.time`. Do not relabel it sensor-to-database latency unless a separate run proves the source/laptop UTC timestamp is correlated closely enough with the server clock.

The post-run summarizer also reports sample SD, min, max, RSSI, SNR, frequency counts, sequence/FCnt gaps, measurement counts, Fabric outbox states, and per-node host/container resource statistics.

What remains physical

Evidence capture, exports, statistics, hashing, most attack generators, and server/network fault injection can be automated. Human action remains only where the experimental condition itself is physical, such as reconnecting actually disconnected hardware, deliberately moving nodes/antennas, or changing a real environmental sensor condition.

A physical reset can be timestamped with `mark`. If a test later proves a safe software-reset mechanism for the exact board mode, a test-specific driver can automate that reset as well.

Canonical source: [deployment/technology-guides/20-rui3-security-node.md](#). For any mutable production-status statement, check the dated baseline and verified live state.